

PROJECT REPORT
On
“Real-Time Leaderboard System”

[Python Programming Post-Lab]

Submitted By:

Muneebur Rahman [CS25D010]

Guided By:

Mr. Yogesh Katre



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

**S. B. JAIN INSTITUTE OF TECHNOLOGY
MANAGEMENT AND RESEARCH, NAGPUR.**

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2025-2026

© S.B.J.I.T.M.R Nagpur 2026

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SESSION 2025-2026

CERTIFICATE

This is to certify that the Mini Project titled “**Real-Time Leaderboard System**” is a bonafide work of **Muneebur Rahman** carried out for the partial fulfillment of course work of “**Python Programming (N-SECCS401P)**”, 4th Semester, Bachelor of Technology in **Computer Science & Engineering**, S. B. Jain Institute of Technology, Management & Research, Nagpur.

Mr. Yogesh Katre
Assistant Professor
(Project Guide)

Dr. Anup Gade
Head of Department

INDEX

CERTIFICATE	i
INDEX	ii
LIST OF FIGURES	iii
CHAPTER 1 INTRODUCTION	1
CHAPTER 2 METHODOLOGY	2-3
CHAPTER 3 TOOLS/PLATFORMS	4
CHAPTER 4 DESIGN & IMPLEMENTATION	4-7
4.1 ALGORITHM	
4.2 FLOWCHART	
4.3 SOURCE CODE	
CHAPTER 5 RESULT & DISCUSSION	8-11
5.1 OUTPUT	
5.2 DISCUSSION	
5.3 APPLICATION	
CHAPTER 6 CONCLUSION	12
REFERENCES	13

LIST OF FIGURES

FIG. NO.	TITLE OF FIGURE	PAGE NO.
1	Fig 4.2.1 Leaderboard flowchart	4
2	Fig 5.1.1 First page UI	8
3	Fig 5.1.2 Adding Players	8
4	Fig 5.1.3 Adding more players, ranks are displayed	9
5	Fig 5.1.4 Searching players	9
6	Fig 5.1.5 List of all players	9

CHAPTER 1

INTRODUCTION

A Real-Time Leaderboard System is a dynamic ranking system used in applications such as gaming platforms, coding competitions, and sports analytics to display user rankings instantly based on their scores or performance.

The main objective of this project is to design an efficient leaderboard using **Heap Data Structures** and **Hashing Techniques**. Traditional sorting methods are inefficient for real-time updates, whereas heaps allow fast retrieval of top-ranked users, and hashing ensures quick access to user data.

Features:

- Real-time score updates
- Efficient ranking system
- Fast insertion and deletion
- Retrieval of top-k players
- Scalable for large datasets

Background:

Leaderboards are widely used in platforms like gaming apps and coding contests where rankings frequently change. Efficient data structures are required to maintain performance and speed.

CHAPTER 2

METHODOLOGY

METHODOLOGY

The system is developed using a combination of **Max Heap** and **Hash Map**.

Steps followed:

1. Store user scores in a hash map (dictionary in Python).
2. Maintain a max heap to track top scores efficiently.
3. When a user score is updated:
 - Update hash map
 - Reinsert into heap
4. Extract top players using heap operations.
5. Handle duplicate entries using lazy deletion technique.

Approach:

- Hashing ensures **$O(1)$** lookup time.
- Heap ensures **$O(\log n)$** insertion and deletion.
- Combined approach ensures optimal performance for real-time systems.

CHAPTER 3

TOOLS/PLATFORMS

1.1 SOFTWARE REQUIREMENT

- Programming Language: Python
- IDE: VS Code / PyCharm
- Operating System: Windows
- Libraries Used:
 - heap (for heap implementation)
 - collections (optional)

DESIGN & IMPLEMENTATION

4.1 ALGORITHM

- Initialize a hash map to store user scores
- Initialize a max heap
- For each operation:
 - If new user → add to hash map
 - If existing user → update score
 - Push updated score into heap
- For leaderboard:
 - Extract top K elements from heap
 - Display rankings

4.2 FLOWCHART

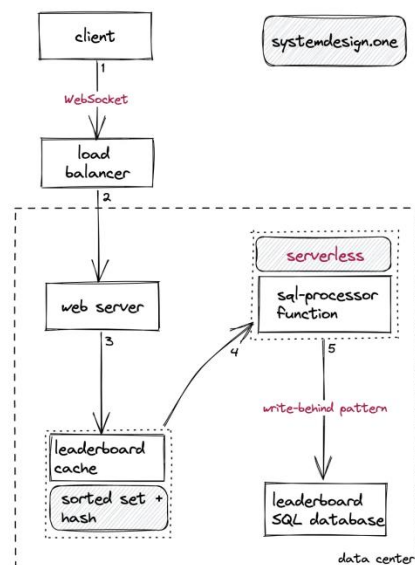


Fig 4.2.1

4.3 SOURCE CODE

```
import tkinter as tk
from tkinter import ttk, messagebox

# Store data
scores = {}

def add_score():
    name = name_entry.get()
    try:
        score = int(score_entry.get())

        if name in scores:
            scores[name] += score
        else:
            scores[name] = score

        update_table()

        name_entry.delete(0, tk.END)
        score_entry.delete(0, tk.END)

    except:
        messagebox.showerror("Error", "Enter valid score")

def update_table(data=None):
    # clear table
    for row in tree.get_children():
        tree.delete(row)
```

```

# use given data or full data
display_data = data if data else sorted(scores.items(), key=lambda x: x[1], reverse=True)

# insert into table
for i, (name, score) in enumerate(display_data):
    tree.insert("", "end", values=(i+1, name, score))

def search_player():
    keyword = search_entry.get().lower()

    filtered = [(name, score) for name, score in scores.items() if keyword in name.lower()]

    # sort filtered data
    filtered = sorted(filtered, key=lambda x: x[1], reverse=True)

    update_table(filtered)

def reset_table():
    search_entry.delete(0, tk.END)
    update_table()

def clear_all():
    scores.clear()
    update_table()

# Window
root = tk.Tk()
root.title("Leaderboard with Search")
root.geometry("420x400")

```

```

# Input
tk.Label(root, text="Name").pack()
name_entry = tk.Entry(root)
name_entry.pack()

tk.Label(root, text="Score").pack()
score_entry = tk.Entry(root)
score_entry.pack()

tk.Button(root, text="Add Score", command=add_score).pack(pady=5)

# Search
tk.Label(root, text="Search Player").pack()
search_entry = tk.Entry(root)
search_entry.pack()

tk.Button(root, text="Search", command=search_player).pack(pady=3)
tk.Button(root, text="Reset", command=reset_table).pack(pady=3)

tk.Button(root, text="Clear All", command=clear_all).pack(pady=5)

# Table
columns = ("Rank", "Name", "Score")
tree = ttk.Treeview(root, columns=columns, show="headings")

for col in columns:
    tree.heading(col, text=col)

tree.pack(pady=10)

root.mainloop()

```

RESULT & DISCUSSION

5.1 OUTPUT

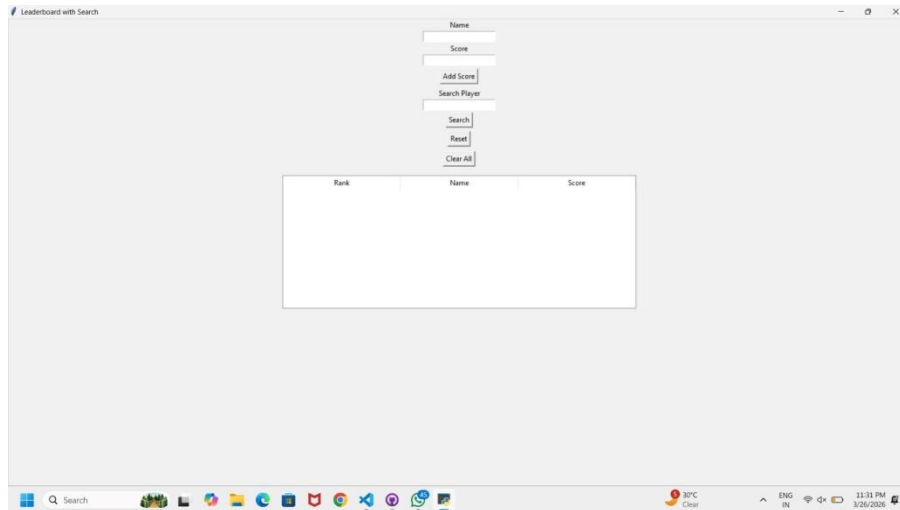


Fig 5.1.1

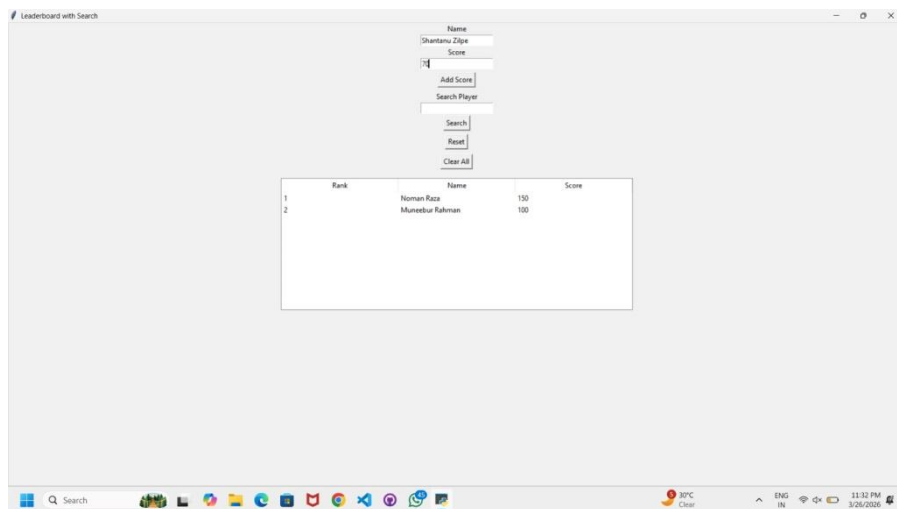


Fig 5.1.2

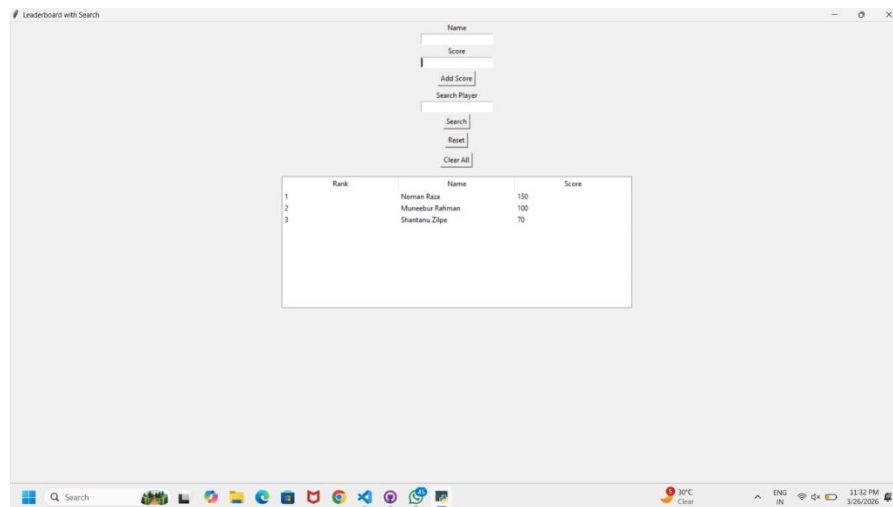


Fig 5.1.3

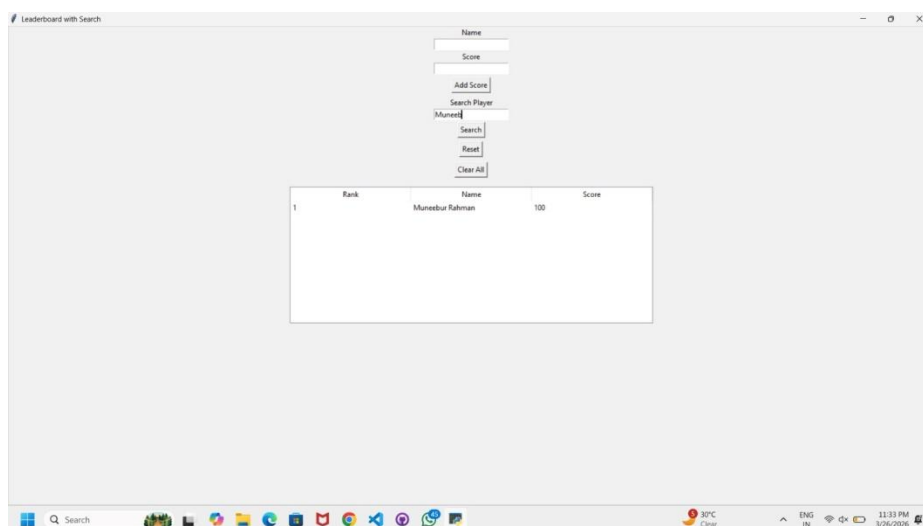


Fig 5.1.4



Fig 5.1.5

5.2 DISCUSSION

The Real-Time Leaderboard System developed using Python demonstrates an efficient and scalable approach to maintaining rankings dynamically. By combining the concepts of heaps and hashing, the system overcomes the limitations of traditional sorting techniques, which are inefficient for real-time updates.

One of the major strengths of this system is its ability to handle frequent updates efficiently. In many real-world applications such as gaming and competitions, scores are updated continuously. If traditional sorting methods were used, the system would need to re-sort the entire dataset after every update, resulting in high time complexity. However, by using a heap data structure, the system ensures that insertion and retrieval operations are performed in logarithmic time, making it significantly faster.

The use of hashing (dictionary in Python) provides constant-time complexity for accessing and updating user scores. This ensures that any change in a player's score is immediately reflected in the system without delay. The combination of hashing and heap results in a highly optimized system where both update and retrieval operations are efficient.

Another important aspect of the system is the use of a **lazy deletion technique**. Since heaps do not support direct updates, duplicate entries may occur when a player's score is updated. To handle this, the system validates entries while extracting them, ensuring that only the latest and correct scores are considered. This improves accuracy without compromising performance.

During the development process, several challenges were encountered, such as handling duplicate entries in the heap, maintaining synchronization between the heap and hash map, and ensuring correct ranking order. These challenges were addressed by implementing validation checks and efficient data handling mechanisms.

Despite its advantages, the system has some limitations. It is currently implemented as a console-based application, which may not be user-friendly for non-technical users. Additionally, it does not include persistent storage, meaning the data is lost once the program is terminated. These limitations can be addressed in future enhancements.

Overall, the system performs efficiently and meets its objectives. It demonstrates the practical application of data structures in solving real-world problems and provides a strong foundation for further improvements.

5.3 APPLICATION

The Real-Time Leaderboard System has a wide range of practical applications in both everyday life and professional environments, especially in areas where ranking, competition, and performance tracking are essential.

One of the primary applications of this system is in **online gaming platforms**. In modern multiplayer games, players compete with each other, and their scores or achievements are continuously updated. A real-time leaderboard helps display the top players instantly, motivating users to improve their performance. It enhances user engagement and creates a competitive environment by showing rankings dynamically without delay.

Another important application is in **coding competition platforms** such as hackathons and online judges. Websites that host programming contests require efficient ranking systems to display participants' scores based on problem-solving performance. Since submissions and scores are updated frequently, a real-time leaderboard ensures accurate and fast ranking updates. This helps participants track their position instantly and encourages healthy competition among programmers.

The system is also widely used in **sports ranking systems**. In tournaments, leagues, or events, teams and players are ranked based on their performance, points, or scores. A real-time leaderboard allows organizers and viewers to monitor rankings as the game progresses. This is particularly useful in live events where rankings change frequently, providing an engaging experience for audiences.

In the field of **e-learning platforms**, leaderboard systems are used to track student performance. Many educational platforms incorporate quizzes, assignments, and gamified learning modules. A leaderboard helps rank students based on their scores, encouraging active participation and improving learning outcomes. It also helps instructors analyze student performance and identify top performers.

Another growing application is in **social media and engagement platforms**. Leaderboards can be used to rank users based on activity levels such as likes, shares, comments, or content contributions. This encourages users to be more active and engaged on the platform. Influencers, content creators, and active users can be recognized through such ranking systems.

Additionally, the system can be applied in **corporate environments** for employee performance tracking, sales leaderboards, and productivity monitoring. Companies can use leaderboard systems to motivate employees by ranking them based on performance metrics, thereby increasing efficiency and competitiveness within teams.

Overall, the Real-Time Leaderboard System provides a fast, efficient, and scalable solution for ranking and performance tracking across various domains. Its ability to handle continuous updates and instantly reflect changes makes it highly valuable in real-world applications.

CONCLUSION

The Real-Time Leaderboard System successfully demonstrates the use of advanced data structures such as heaps and hashing to efficiently manage dynamic rankings. The project achieves its primary objective of maintaining a real-time leaderboard where scores are continuously updated and rankings are displayed instantly.

The implementation highlights the importance of selecting appropriate data structures for solving specific problems. By using a hash map, the system ensures fast access and updates of user data, while the heap structure enables efficient retrieval of top-ranked players. This combination results in a highly optimized system with improved performance compared to traditional approaches.

The project also provides valuable insights into algorithm design and optimization. It shows how time complexity can be reduced by avoiding unnecessary operations such as repeated sorting. The use of techniques like lazy deletion further enhances the efficiency and correctness of the system.

From a learning perspective, this project helps in understanding key programming concepts such as data structures, algorithm design, and problem-solving strategies. It also demonstrates how theoretical concepts can be applied to build practical applications.

Although the current implementation is effective, there is scope for further improvement. The system can be enhanced by adding a graphical user interface (GUI) to improve usability. Additionally, integrating a database can enable permanent storage of data, allowing the system to retain information even after restarting. Features such as user authentication, real-time synchronization using APIs, and cloud-based storage can also be added to make the system more robust and scalable.

In conclusion, the Real-Time Leaderboard System is a practical and efficient solution for managing rankings in dynamic environments. It serves as a strong foundation for developing more advanced systems and demonstrates the power of data structures in real-world applications.

REFERENCE

- **Introduction to Algorithms –**
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein
(Used for understanding Heap and data structures concepts)

- **Data Structures and Algorithms in Java –**
Michael T. Goodrich, Roberto Tamassia
(Reference for Java-based data structures implementation)

- **Java Collections Framework –**
Oracle Documentation
<https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
(Used for PriorityQueue and HashMap)

- **Design A Leaderboard –**
<https://leetcode.com/problems/design-a-leaderboard/>
(Problem inspiration and logic reference)

- **Heap Data Structure –**
<https://www.geeksforgeeks.org/heap-data-structure/>

- **Hash Table –**
<https://www.geeksforgeeks.org/hashing-data-structure/>

- **GeeksforGeeks –**
<https://www.geeksforgeeks.org/>
(Used for understanding concepts and examples)

- **W3Schools –**
<https://www.w3schools.com/>
(Basic Java reference)